

AULA 10 • PROGRAMAÇÃO ORIENTADA A OBJETOS

Collections: Set e Map

Guardar valores **sem repetição** com `Set` e relacionar `chave` → `valor` com `Map` — as duas coleções que faltavam.

Curso de Java • P00 • Prof. Rob Silva

O caminho de hoje.

- 01 Recap rápido: List da Aula 9
- 02 `Set` — coleção sem duplicatas
- 03 `Map` — chave → valor
- 04 Contar ocorrências com `getOrDefault`
- 05 Aplicando no estoque da MegaStore (JAVA-104)
- 06 Tarefa da aula — ticket JAVA-110

Na Aula 9 vimos List. Hoje, Set e Map.

- Vocês usaram `List` / `ArrayList`: mantém ordem e **aceita duplicatas**, percorrido com `for` e ordenado com `Collections.sort()`.
- `Set`: como uma lista, mas **não guarda valores repetidos**.
- `Map`: guarda pares `chave` → `valor` — você busca pelo valor a partir da chave.

Set = conjunto, sem repetição.

- Adicionar um valor que já existe é simplesmente **ignorado**.
- Bom para responder "**quais** existem?" sem contar repetidos.
- Operações úteis: `add`, `contains` e `size`.



HashSet: a duplicata é ignorada.

```
import java.util.HashSet;
import java.util.Set;

Set<String> categorias = new HashSet<>();
categorias.add("Eletrônicos");
categorias.add("Eletrônicos"); // ignorado: sem duplicata
categorias.add("Periféricos");

System.out.println(categorias.size());           // 2
System.out.println(categorias.contains("Eletrônicos")); // true
```

Mesmo adicionando "Eletrônicos" duas vezes, o `size()` é **2**. O `contains` responde se um valor está dentro.

HashSet ou TreeSet? Depende da ordem.

HashSet

Sem ordem garantida ao percorrer. É o mais rápido — use quando a ordem não importa.

TreeSet

Mantém os valores em **ordem alfabética** (ou numérica) automaticamente.

```
Set<String> ordenado = new TreeSet<>(categorias);  
// TreeSet mantém em ordem alfabética  
for (String c : ordenado) {  
    System.out.println(c);  
}
```

Map = chave → valor.

- Cada **chave** é única e aponta para um **valor**.
- Como uma agenda: o nome (chave) leva ao telefone (valor).
- Você busca pelo valor a partir da chave — sem percorrer tudo.



Uma agenda de contatos com put e get.

```
import java.util.HashMap;
import java.util.Map;

Map<String, String> agenda = new HashMap<>();
agenda.put("Ana", "9999-1111");
agenda.put("Bruno", "9888-2222");

System.out.println(agenda.get("Ana"));           // 9999-1111
System.out.println(agenda.containsKey("Bruno")); // true
```

`put(chave, valor)` grava o par; `get(chave)` devolve o valor. `Map<String, String>`: chave e valor são texto.

Percorrer um Map: keySet e entrySet.

```
// chaves
for (String nome : agenda.keySet()) {
    System.out.println(nome + " -> " + agenda.get(nome));
}

// pares (entrySet)
for (Map.Entry<String, String> e : agenda.entrySet()) {
    System.out.println(e.getKey() + ": " + e.getValue());
}
```

`keySet()` dá as chaves; `entrySet()` dá os pares prontos, com `getKey()` e `getValue()`.

PADRÃO • CONTAGEM

Contar é um padrão.

Quantas vezes cada valor aparece? Um `Map` guarda a chave e vai somando o contador. É a mesma receita para palavras, votos ou produtos por categoria.

getOrDefault evita o null e conta em uma linha.

```
String[] palavras = {"sim", "nao", "sim", "sim", "nao"};
Map<String, Integer> contagem = new HashMap<>();

for (String palavra : palavras) {
    contagem.put(palavra,
        contagem.getOrDefault(palavra, 0) + 1);
}
System.out.println(contagem); // {sim=3, nao=2}
```

`getOrDefault(chave, 0)` devolve o contador atual ou **0** se a chave ainda não existe — aí somamos 1 e regravamos.

HashMap ou TreeMap? De novo, é sobre ordem.

Sem ordem garantida das chaves. É o mais rápido — o padrão para a maioria dos casos.

Mantém as chaves em **ordem alfabética** (ou numérica) ao percorrer.

Mesma ideia do Set: trocar `new HashMap<>()` por `new TreeMap<>()` já entrega o relatório ordenado pela chave. Os métodos `put`, `get` e `getOrDefault` são idênticos.

Trazendo Set e Map para o estoque da MegaStore.

- Cada `Produto` tem uma `categoria` em texto — e várias se repetem no estoque.
- **Set** de categorias: a lista de categorias **sem repetir** — só os nomes únicos.
- **Map** de contagem: para cada categoria, **quantos produtos** existem nela.

É o `Produto` do seu sistema (`JAVA-104`): `nome`, `preco`, `quantidade` e `categoria`. Agora resumimos o estoque por categoria.

Contando produtos por categoria.

```
Map<String, Integer> porCategoria = new HashMap<>();
for (Produto p : estoque) {
    String cat = p.getCategoria();
    porCategoria.put(cat,
        porCategoria.getOrDefault(cat, 0) + 1);
}
// {Eletrônicos=4, Periféricos=2}
```

É exatamente a receita do slide da contagem — só trocamos palavras por `p.getCategoria()`. A chave é a categoria; o valor, quantos produtos nela.

Três ideias para levar para casa.

Set

Coleção sem duplicatas. `HashSet` não garante ordem; `TreeSet` mantém ordenado.

Map

Pares `chave → valor`. `put` / `get`, percorrido com `keySet` ou `entrySet`.

Contagem

`getOrDefault(chave, 0) + 1` conta ocorrências sem cair em `null`.

JAVA-110

MÉDIO

TO DO

BACKEND

RESPONSÁVEL: VOCÊ

Resumo de categorias no relatório de estoque

HISTÓRIA DO USUÁRIO

Como gerente da MegaStore, quero ver no relatório **quantos produtos existem por categoria** e a **lista de categorias sem repetição**, para entender a distribuição do estoque.

CONTEXTO

Evolui o sistema do JAVA-104 : o relatório atual lista produtos. Adicione um resumo com as categorias únicas (Set) e a contagem de produtos por categoria (Map).

JAVA-110

MÉDIO

PRONTO QUANDO TODOS OS ITENS PASSAREM

CRITÉRIOS DE ACEITE

- ☒ Usar um `Set<String>` para coletar as categorias **sem duplicatas**.
- ☒ Usar um `Map<String, Integer>` com `getOrDefault` para contar produtos por categoria.
- ☒ Imprimir o resumo na **opção 3 (Relatório)**, percorrendo o Map com `keySet` ou `entrySet`.
- ☒ Usar `TreeSet` / `TreeMap` se quiser o resultado em ordem alfabética.

DESAFIO EXTRA

Use um `Map<String, Double>` para somar o **valor total em estoque** (`preco × quantidade`) por categoria — opcional.

#megastore

#java-104

#set-map

#entrega: próxima aula

Resolva usando apenas `Set` (`HashSet/TreeSet`) e `Map` (`HashMap/TreeMap`) com iteração – nada além do visto na Aula 10.

